

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

图与树的基础概念

Fundamentals of Graphs and Trees

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. 图的基本概念
2. 树的基本概念
3. 图的存储方式
4. 总结



目录

1. 图的基本概念
2. 树的基本概念
3. 图的存储方式
4. 总结

什么是图

图（Graph）由两个集合组成：

$$G = (V,\ E)$$

- V：**顶点集**（Vertex），图中的“点”——可以是城市、人、状态
- E：**边集**（Edge），连接两个顶点的“线”，表示两者有关系
- 记 $n = |V|$ **为顶点数**，$m = |E|$ **为边数**
- 边可以带**权值**（距离、花费、时间），也可以不带

图 = 一堆点 + 点与点之间的连接关系

简单图与多重图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

简单图（Simple Graph）

- **无自环**：不存在自己连自己的边
- **无重边**：任意两点之间至多一条边
- 算法竞赛中，除非特别说明，一般讨论简单图
- 简单无向图最多有 $\frac{n(n-1)}{2}$ 条边

多重图（Multigraph）

- 允许两点之间存在多条边（重边）
- 允许自己连自己（自环）
- 生活中常见：城市间的多条航班
- 存储时需要用额外空间记录重边

本课默认讨论简单图，概念更清晰，也覆盖绝大多数场景

有向图与无向图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

无向图（Undirected）

- 边没有方向，(u, v) 与 (v, u) 是同一条边
- 例子：朋友关系、双向道路
- **度**：与该点相连的边数
- 无向图中所有点的度之和 $= 2m$

有向图（Directed）

- 边有方向，$u \to v$ 与 $v \to u$ 是两条边
- 例子：微博关注、单向道路
- **出度**：出去的边数；**入度**：进来的边数
- 有向图中出度之和 $=$ 入度之和 $= m$

无论哪种图，都需要先完整地存进内存才能处理

路径、环与连通性

- **路径（Path）**：从点 u 出发，沿边依次走到点 v 的顶点序列
- **路径长度**：路径上经过的**边数**（不是点数）
- **环（Cycle）**：起点与终点相同的路径
- **连通（Connected）**：无向图中任意两点之间都存在路径
- **连通块（连通分量）**：极大的连通子图，块与块之间没有任何边

连通性是图的基本性质；不连通的图可分成多个连通块分别研究

目录

1 图的基本概念

2 树的基本概念

3 图的存储方式

4 总结

树：一种特殊的图

树（Tree）：一个**无环**且**连通**的**无向图**。

- n 个点的树恰好有 $n-1$ 条边
- 树中任意两点之间有且仅有**一条**简单路径
- 任意加一条边 $\Rightarrow$ 必出现一个环
- 任意删一条边 $\Rightarrow$ 不再连通，分成两棵树

树是最简单的连通图，也是算法中最常见的结构之一

树的常用术语（一）

- **根（Root）**：人为指定的起点
- **父节点（Parent）**：某节点上一层直接相连的节点
- **子节点（Child）**：某节点下一层直接相连的节点
- **兄弟节点（Sibling）**：拥有同一个父节点的其他节点
- **叶子（Leaf）**：没有子节点的节点
- **深度（Depth）**：根到该节点经过的边数

无根树指定根之后，就变成了有根树，上述术语才有意义。

树的常用术语（二）

- **祖先（Ancestor）**：从根到该节点路径上的所有节点（不含自身）
- **后代（Descendant）**：以该节点为根的子树中的所有节点（不含自身）
- **子树（Subtree）**：某节点及其所有后代构成的树
- **高度（Height）**：从该节点到最远叶子的边数；树的高度是根的高度
- **层（Level）**：根为第 0 层，其子为第 1 层，依此类推
- **节点的度**：有根树中通常指子节点的个数

图、树与森林的关系

概念	条件	直观理解
无向图	无附加条件	点 + 边，可以有环，也可以不连通
连通图	任意两点有路径	整张图“连成一片”
树	连通 + 无环	边数最少的连通图（$n-1$ 条）
森林	无环	若干棵树拼在一起，可以有多块

不连通的图，每个连通块可以单独分析

目录

1 图的基本概念

2 树的基本概念

3 图的存储方式

4 总结

图的三种存储方式

方式	空间	判断 u–v 有边	遍历 u 的邻居
邻接矩阵	$O(n^2)$	$O(1)$	$O(n)$
邻接表	$O(n+m)$	$O(\deg(u))$	$O(\deg(u))$
链式前向星	$O(n+m)$	$O(\deg(u))$	$O(\deg(u))$

其中 n 为点数，m 为边数，$\deg(u)$ 为点 u 的度数。

选哪种？看数据范围：$n \le 10^3$ 随便，$n \ge 10^4$ 用邻接表

存储方式一：邻接矩阵

用二维数组 $g[u][v]$ 表示 u 与 v 之间是否有边。

- **无向图**：$g[u][v] = g[v][u] = 1$
- **有向图**：只令 $g[u][v] = 1$
- **带权图**：$g[u][v] = w$，无边记 ∞
- **优点**：$O(1)$ 查边，写法最简单
- **缺点**：空间 $O(n^2)$，稀疏图严重浪费

5 个点的无向图，矩阵形如：

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

无向图的矩阵是**对称**的，对角线一般为 0。

代码：邻接矩阵建图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1005;
int g[MAXN][MAXN];              // 邻接矩阵，下标 1..n

int main() {
    int n, m;
    cin >> n >> m;

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        g[u][v] = w;            // 有向图只写这一行
        g[v][u] = w;            // 无向图再补反向
    }

    // O(1) 判断 u、v 之间是否有边
```

代码：邻接矩阵建图


```
19      int u = 1, v = 3;
20      if (g[u][v] != 0)
21          cout << "边存在，权值为 " << g[u][v] << "\n";
22
23      // 遍历 u 的所有邻居：O(n)
24      for (int v = 1; v <= n; v++)
25          if (g[u][v] != 0)
26              cout << u << " 的邻居：" << v << "\n";
27      return 0;
28  }
```

存储方式二：邻接表（vector）

只存**实际存在的边**：$g[u]$ 是一个列表，装着所有与 u 相邻的点。

- 空间 $O(n+m)$，对稀疏图非常友好
- 遍历 u 的邻居：直接扫 $g[u]$，$O(\deg(u))$
- 判断 u–v 有边：要在 $g[u]$ 中查找，$O(\deg(u))$
- C++ 写法：`vector<vector<int>> g(n+1);`
- 带权边：把 `int` 换成 `pair<int,int>` 或结构体

竞赛与工程中最常用的默认选择

代码：邻接表建图与遍历邻居

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;                          // n 个点，m 条边
    vector<vector<int>> g(n + 1);           // 下标使用 1..n

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);                  // 无向图加两条
        g[v].push_back(u);
        // 有向图只写 g[u].push_back(v);
    }

    // 打印每个点的邻居列表
    for (int u = 1; u <= n; u++) {
```

代码：邻接表建图与遍历邻居

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19          cout << u << " -> ";
20          for (int v : g[u]) cout << v << " ";
21          cout << "\n";
22      }
23      return 0;
24  }
```

存储方式三：链式前向星

用数组模拟链表，把同一个起点的所有出边串成一条链。

数组	含义
`head[u]`	以 u 为起点的第一条边的编号（初值 -1）
`to[i]`	第 i 条边的终点
`nxt[i]`	与第 i 条边同起点的下一条边的编号
`w[i]`	第 i 条边的权值（可选）

遍历方式：`for (int i=head[u]; i!=-1; i=nxt[i])`

这是遍历图中邻居的一种高效写法。

空间 $O(n+m)$、常数小，但不如 vector 直观

代码：链式前向星建图


```cpp
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005, MAXM = 200005;   // 无向图要 2m
int head[MAXN], to[MAXM], nxt[MAXM], w[MAXM];
int cnt = 0;                              // 已加入的边数

void addEdge(int u, int v, int weight) {
    to[cnt]  = v;                         // 第 cnt 条边终点是 v
    w[cnt]   = weight;
    nxt[cnt] = head[u];                   // 接在 u 的链头
    head[u]  = cnt;                       // 更新链头
    cnt++;
}

int main() {
    int n, m;
    cin >> n >> m;
```

代码：链式前向星建图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19      memset(head, -1, sizeof(head));
20      for (int i = 0; i < m; i++) {
21          int u, v, wt;
22          cin >> u >> v >> wt;
23          addEdge(u, v, wt);
24          addEdge(v, u, wt);                  // 无向图反向再加一条
25      }
26      // 遍历点 u 的所有出边
27      int u = 1;
28      for (int i = head[u]; i != -1; i = nxt[i])
29          cout << u << " -> " << to[i] << " (w=" << w[i] << ")\n";
30      return 0;
31  }
```

目录

1 图的基本概念

2 树的基本概念

3 图的存储方式

4 总结



总结：概念速查

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

图

- 图 $G = (V, E)$：点 + 边
- 简单图：无自环、无重边
- 有向 / 无向；出度 / 入度
- 路径、环、连通块

树

- 连通 + 无环的无向图
- n 点 $n-1$ 边，两点间路径唯一
- 根、父子、兄弟、叶子、深度
- 祖先、后代、子树、高度、层

存储

- 邻接矩阵：$O(n^2)$，查边 $O(1)$
- 邻接表：$O(n+m)$，最常用
- 链式前向星：$O(n+m)$，常数小

所有存储方式都只服务于同一个动作：
枚举某个点的所有邻居。
后续的图算法，大多都建立在这个基本操作之上。

存储方式怎么选

场景	推荐存储	理由
$n \le 1000$ 的稠密图	邻接矩阵	写法简单，查边 $O(1)$
一般稀疏图	邻接表	空间省，遍历快，代码短
边权复杂 / 追求常数	链式前向星	内存连续，数组可复用

三选一即可，不必全都用；先写对，再写快

下一步学什么

- 理解图与树的基本概念
- 掌握至少一种图的存储方式
- 能够熟练枚举任意点的所有邻居
- 这些是后续学习图算法（如遍历、最短路、生成树等）的共同基础

今天把“图怎么存”弄清楚，下一阶段就能直接开始“图怎么用”

附录：常见陷阱与易错点

- 无向图建边只加了一条：后续处理会直接失效
- 链式前向星忘记 `memset(head, -1, ...)`：遍历越界
- 链式前向星数组开小：无向图需要 $2m$ 条边，只开 m 会 RE
- 顶点编号从 0 还是从 1 开始：混用会导致 vector 越界
- 把“树”当成有向图：树是无向图，根只是人为指定的视角
- 稠密图硬用邻接表：m 很大时 vector 反复扩容，效率下降

附录：术语中英对照

中文	英文	中文	英文
图	Graph	树	Tree
顶点	Vertex / Node	边	Edge
度	Degree	环	Cycle
路径	Path	连通	Connected
叶子	Leaf	深度	Depth
兄弟	Sibling	祖先	Ancestor
后代	Descendant	高度	Height